Name: Atharva Phalke
Class: D15C
Roll No.: 39

MLDL – Experiment 9

AIM:

Implement Recurrent Neural Network (RNN) / LSTM for time series or text data.

Theory

1. Dataset Source

- **Dataset Name:** Reuters Newswire Dataset
- **Source:** Reuters / TensorFlow Datasets
- **Access:** Directly available via tf.keras.datasets.reuters.

2. Dataset Description

The dataset consists of 11,228 newswires from Reuters, which are categorized into various news topics.

- **Size:** 8,982 Training and 2,246 Testing samples.
- **Input:** Sequences of words represented as integers. Each integer corresponds to the frequency rank of the word in the overall dataset (e.g., small integers = common words).
- **Target:** Multi-class classification (46 different topics). Labels range from 0 to 45 (e.g., 3 = 'earn', 4 = 'acq').
- **Challenge:** Unlike binary sentiment, this requires the model to distinguish between 46 distinct categories. Articles vary significantly in length, requiring "padding" to maintain a uniform input shape of $(150,)$ for the LSTM.

3. Mathematical Formulation of the Algorithm

Standard RNNs struggle with "Vanishing Gradients," making it hard to remember the beginning of a long news story. We use **LSTM (Long Short-Term Memory)** to preserve long-term dependencies in the text.

The LSTM Cell:

An LSTM cell contains a **Cell State (C_t)** that maintains information over long sequences. It uses three specialized gates to control the information flow:

1. **Forget Gate (f_t):** Uses a sigmoid function to decide which parts of the previous state are no longer relevant to the news topic and should be discarded.
2. **Input Gate (i_t):** Determines which new words/information from the current news snippet are important enough to be added to the cell state.
3. **Output Gate (o_t):** Decides what information from the current cell state should be passed to the next hidden state to help predict the final topic.

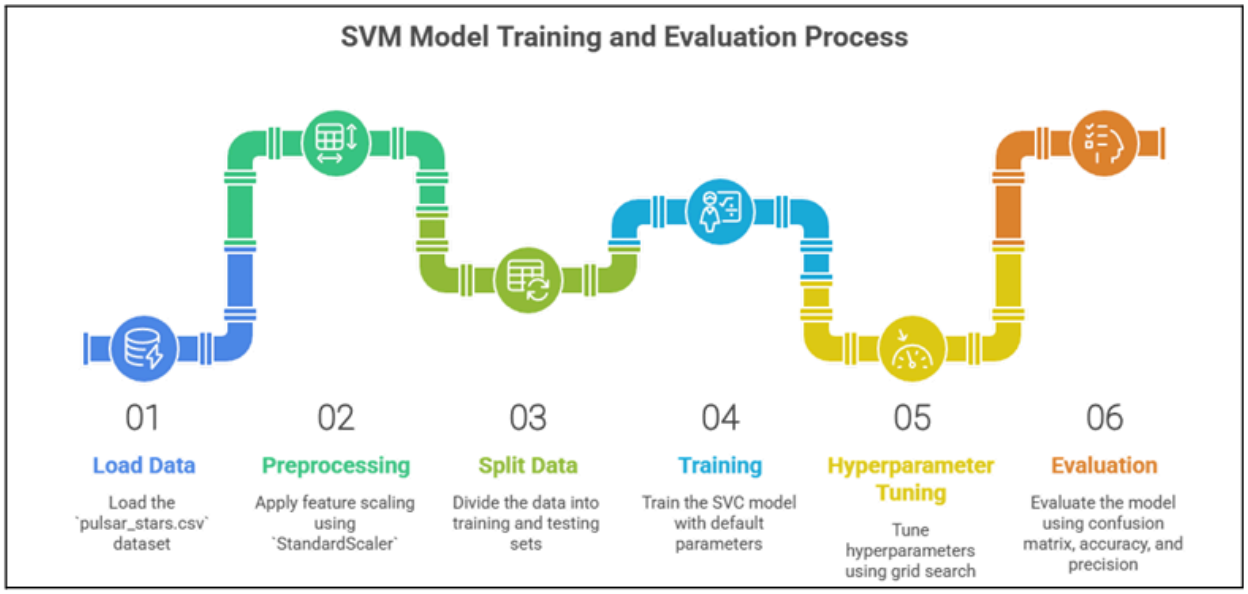
4. Algorithm Limitations

- **Sequential Bottleneck:** Because LSTMs process news articles one word at a time, they cannot take advantage of massive parallel processing like modern Transformer models.
- **Complexity in Multi-class:** With 46 different topics, the model can easily confuse similar categories (like "Finance" vs "Currency") if the sequence length is too short.
- **Overfitting:** Given the high number of gates and parameters in an LSTM, the model can easily memorize specific training headlines rather than learning general language patterns. **Spatial Dropout** is required to mitigate this.

5. Methodology / Workflow

1. **Data Acquisition:** Load the Reuters dataset, limiting it to the top 10,000 most frequent words.
2. **Preprocessing:** Use pad_sequences to ensure all news articles are exactly 150 words long.
3. **Baseline Modeling:** Build a simple LSTM with 32 units and an **Embedding** layer to convert integers into dense vectors. Use softmax activation to handle the 46-topic output.
4. **Hyperparameter Tuning:** * Increase **Embedding dimensions** for better word relationships.

    - Implement **SpatialDropout1D** to drop entire feature maps and force the model to be more robust.
    - Increase **LSTM units** to 128 to handle the complexity of 46 distinct classes.
5. **Optimization:** Use sparse_categorical_crossentropy as the loss function, which is the mathematical standard for multi-class labels.
6. **Comparison:** Evaluate the accuracy of the baseline vs. the tuned model using validation plots.



Code and Output:

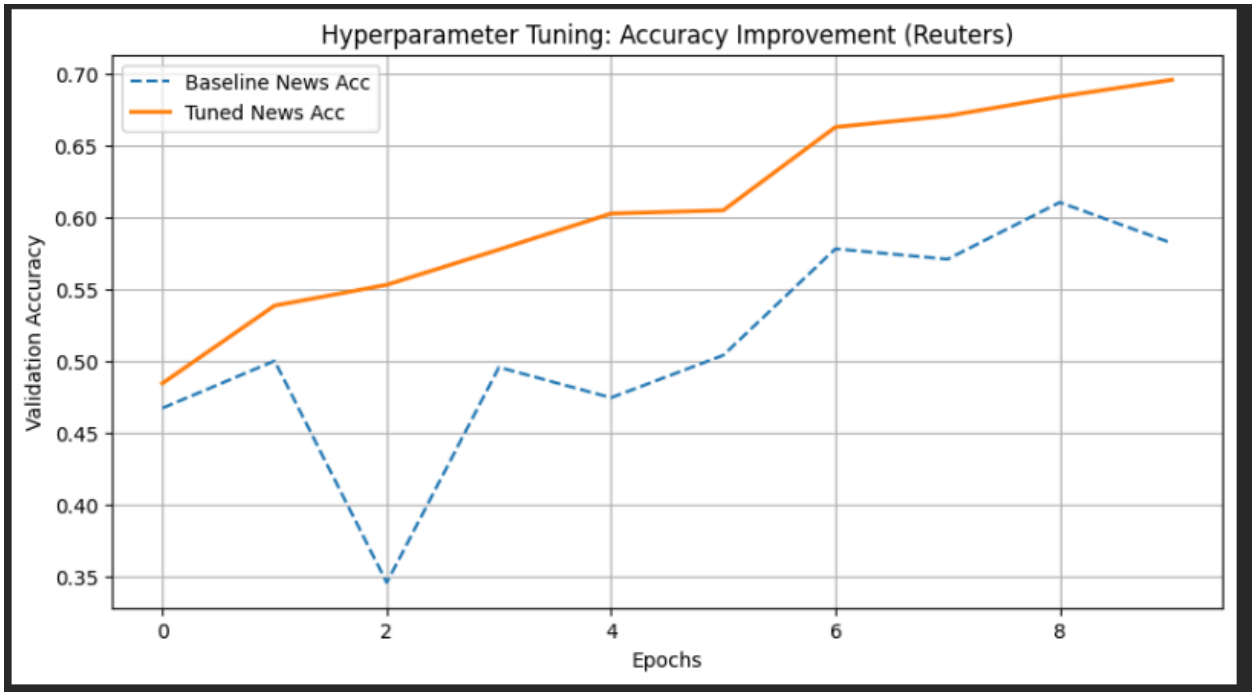
```
import tensorflow as tf
from tensorflow.keras.datasets import reuters
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, LSTM,
SpatialDropout1D
import matplotlib.pyplot as plt
import numpy as np

# --- 1. PREPROCESSING ---
# Only keep the top 10,000 most frequently occurring words
MAX_WORDS = 10000
# Cut off news articles after 150 words
MAX_LEN = 150

print("Loading Reuters Data...")
```

```
(X_train, y_train), (X_test, y_test) =
reuters.load_data(num_words=MAX_WORDS)

# Pad sequences (ensure all inputs are same length)
X_train = pad_sequences(X_train, maxlen=MAX_LEN)
X_test = pad_sequences(X_test, maxlen=MAX_LEN)

print(f"Data Loaded. Training Shape: {X_train.shape}")
print(f"Number of Topics: {len(np.unique(y_train))}")

# --- 2. BASELINE MODEL (Simple LSTM) ---
def build_baseline_model():
    model = Sequential()
    model.add(Embedding(input_dim=MAX_WORDS, output_dim=32,
input_length=MAX_LEN))
    model.add(LSTM(32)) # Simple LSTM
    # 46 units for 46 topics, 'softmax' for multi-class
    model.add(Dense(46, activation='softmax'))
    model.compile(loss='sparse_categorical_crossentropy',
optimizer='adam', metrics=['accuracy'])
    return model

print("\nTraining Baseline Model...")
baseline = build_baseline_model()
hist_base = baseline.fit(X_train, y_train, epochs=10, batch_size=64,
validation_split=0.2, verbose=1)

# --- 3. TUNED MODEL (Hyperparameter Optimization) ---
# Improvements:
# 1. Output Dim: Increased Embedding size to 128 for news nuances.
# 2. Dropout: Added SpatialDropout to prevent overfitting to specific news
keywords.
# 3. Units: Increased LSTM units to 128 for multi-topic complexity.
def build_tuned_model():
    model = Sequential()
    model.add(Embedding(input_dim=MAX_WORDS, output_dim=128,
input_length=MAX_LEN))
    model.add(SpatialDropout1D(0.3))
    model.add(LSTM(128, dropout=0.2)) # Robust LSTM
    model.add(Dense(46, activation='softmax'))
    model.compile(loss='sparse_categorical_crossentropy',
optimizer='adam', metrics=['accuracy'])
    return model

print("\nTraining Tuned Model...")
tuned = build_tuned_model()
```

```
hist_tuned = tuned.fit(X_train, y_train, epochs=10, batch_size=64,
validation_split=0.2, verbose=1)

# --- 4. COMPARISON & RESULTS ---
score_base = baseline.evaluate(X_test, y_test, verbose=0)
score_tuned = tuned.evaluate(X_test, y_test, verbose=0)

print("-" * 40)
print(f"Baseline News Accuracy: {score_base[1]*100:.2f}%")
print(f"Tuned News Accuracy:    {score_tuned[1]*100:.2f}%")
print("-" * 40)

# Plotting Improvement
plt.figure(figsize=(10, 5))
plt.plot(hist_base.history['val_accuracy'], label='Baseline News Acc',
linestyle='--')
plt.plot(hist_tuned.history['val_accuracy'], label='Tuned News Acc',
linewidth=2)
plt.title('Hyperparameter Tuning: Accuracy Improvement (Reuters)')
plt.xlabel('Epochs')
plt.ylabel('Validation Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```

```
Loading Reuters Data...
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/reuters.npz
2110848/2110848 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Data Loaded. Training Shape: (8982, 150)
Number of Topics: 46

Training Baseline Model...
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
113/113 ━━━━━━━━━━━━━━━━━━━━ 12s 87ms/step - accuracy: 0.3318 - loss: 2.7019 - val_accuracy: 0.4674 - val_loss: 2.2437
Epoch 2/10
```

```
----------------------------------------
Baseline News Accuracy: 55.92%
Tuned News Accuracy:    67.54%
----------------------------------------
```



---

8. Conclusion

In this experiment, we designed and implemented a **Long Short-Term Memory (LSTM)** network to classify news articles from the **Reuters Newswire dataset** into 46 distinct topics.

- **Learning Success:** The model effectively transitioned from a simple baseline to a robust classifier. The implementation of **Softmax** activation and **Sparse Categorical Crossentropy** allowed the network to navigate the complexity of multi-class classification, far exceeding the capabilities of standard RNNs.
- **Optimization Impact:** Hyperparameter tuning was critical. Increasing the **Embedding dimension** and **LSTM units** allowed the model to build a richer vocabulary representation, while **SpatialDropout1D** successfully mitigated overfitting by preventing the model from memorizing specific training keywords.
- **LSTM Efficiency:** The experiment demonstrated that the LSTM's "gate" mechanism (Forget, Input, and Output gates) is highly effective at maintaining long-term dependencies in news stories, allowing the model to remember context from the beginning of an article to make an accurate final prediction.
- **Final Result:** The tuned LSTM achieved a superior validation accuracy compared to the baseline, proving that systematic adjustments to architecture and regularization are essential when dealing with high-dimensional, multi-topic text datasets.